Week 8 슬라이드 자습 가이드

Virtual Memory to Physical Memory (Kernel View): Page Tables, TLB, Page Fault, Buddy Allocator, SLUB · Sungyong Park, Ph.D. (DISCOS Lab, Sogang University)

이 자료를 사용하는 방법

이 자료는 Week 8 슬라이드(총 57쪽)를 보면서 함께 읽는 자습 가이드입니다. 슬라이드는 영어로, 설명은 한국어로 작성했습니다. 슬라이드마다 내용 요약과 기억할 점을 정리했습니다. 1, 3, 23, 36, 57쪽은 표지, 구간 안내, 마지막 슬라이드라서 표에 넣지 않았습니다.

이번 주 학습 목표. 수업이 끝나면 다음 세 가지를 말로 설명할 수 있어야 합니다.

1. 가상 주소가 page table walk와 TLB를 거쳐 물리 주소가 되는 과정과 그 비용
2. page fault의 종류(not-present와 permission, minor와 major), 처리 흐름, 새 물리 frame이 필요한 경우
3. 커널이 물리 페이지(buddy allocator)와 작은 객체(SLUB)를 할당하는 방식, 그리고 /proc으로 관찰하는 방법

수업 구성. 세 가지 질문으로 진행합니다. 1부는 가상 주소가 어떻게 물리 주소가 되는지(4–22쪽), 2부는 page fault가 나면 커널이 무엇을 하는지(24–35쪽), 3부는 커널이 물리 메모리를 어떻게 나눠 주는지(37–54쪽)를 다룹니다. 55쪽은 메모리를 줄일 때의 대상과 trade-off, 56쪽은 한 장 요약입니다.

추천 학습 방법.

1. 슬라이드 왼쪽의 Key Points를 읽고, 아래의 Key Takeaway를 다시 한 번 읽습니다.
2. 이 자료의 요약과 "기억할 점"으로 이해한 내용을 확인합니다.
3. 부가 끝날 때마다 Checkpoint 슬라이드(22, 35, 54쪽)의 질문에 먼저 스스로 답해 보고, 마지막 장의 답과 비교합니다.
4. 슬라이드에 "Week 9"라고 표시된 내용(reclaim, slow path, compaction이 시작되는 조건 등)은 9주차에서 자세히 다룹니다. 이번 주에는 이해하지 못해도 괜찮습니다.

참고 사항. 예시 출력은 Linux 6.x, x86-64 환경에서 얻은 값입니다. 개수와 시간 같은 수치는 예시이며, 커널 버전과 환경에 따라 달라집니다. 주소 변환 설명은 x86-64를 기준으로 하고, ARM64와의 차이는 9쪽에 있습니다.

쪽	슬라이드	내용 요약	기억할 점
2	Roadmap of Part 3	Part 3은 4주 구성입니다. 7주차는 애플리케이션 관점의 가상 메모리, 8주차(이번 주)는 커널 관점의 주소 변환입니다. 8주차에서는 page table과 TLB, page fault(minor/major), buddy와 slab allocator를 다룹니다. 9주차는 memory pressure와 page reclaim(watermark, kswapd와 direct reclaim, LRU, swap과 zram, PSI와 cgroup), 10주차는 reclaim이 따라가지 못할 때의 OOM killer, LMK와 원인 분석을 다룹니다.	이번 주는 "가상 주소가 어떻게 물리 페이지가 되고, 그 물리 페이지를 커널이 어떻게 관리하는가"입니다. 부족해질 때의 일은 9주차입니다.

1부. 가상 주소는 어떻게 물리 주소가 될까요? (4–22쪽)

이 블록의 핵심은 다음과 같습니다. page table이 가상 페이지를 물리 frame에 대응시키고, TLB가 그 번역을 캐시하며, 페이지 크기와 context switch, 다중 core가 이 구조의 비용을 결정합니다.

쪽	슬라이드	내용 요약	기억할 점
4	Recap: From VMA to Physical Memory	Week 7 복습입니다. malloc()/new는 libc를 거쳐 brk()나 mmap()으로 VMA(허용된 주소 범위)를 만들거나 늘립니다. 이때는 물리 페이지가 아직 없습니다. 첫 접근(first touch)에서 page fault가 나면, 커널이 buddy allocator에서 빈 프레임을 받아 page table을 갱신하고 실행을 재개합니다.	VMA는 "허용된 범위"일 뿐 물리 메모리가 아닙니다. 이번 주는 그 뒤의 과정을 다룹니다.
5	How Paging Works	가상 주소 공간은 page(4 KiB)로, 물리 메모리는 같은 크기의 frame으로 나눕니다. 가상 주소는 페이지 번호(VPN)와 offset으로 이루어지고, 프로세스마다 하나인 page table이 VPN을 PFN에 대응시킵니다. 물리 주소 = PFN + 같은 offset입니다. 연속된 page가 비연속 frame을 써도 됩니다.	바뀌는 것은 번호뿐이고 offset은 그대로입니다.

쪽	슬라이드	내용 요약	기억할 점
6	Page Table Structure: The Flat Table and Its Problems	가장 단순한 구조는 VPN으로 찾는 PTE 배열 하나입니다 (PTE 주소 = base + VPN × PTE 크기, base는 CR3/TTBR에 저장). 48비트 주소, 4 KiB 페이지, 8 B PTE이면 프로세스마다 512 GiB가 필요합니다. 문제는 세 가지입니다: 크기, 연속성(배열이 연속 메모리 한 덩어리여야 함), 희소성(대부분 비어 있는데 모두 예약).	문제 세 가지 이름(크기, 연속성, 희소성)을 외워 둡니다. 8 MiB만 매핑해도 쓰이는 PTE는 2,048개입니다.
7	What If We Increase the Page Size?	2 MiB 페이지를 쓰면 PTE 수가 줄어 flat table이 1 GiB로 작아집니다. 하지만 table은 여전히 연속 배열이고 대부분 비어 있습니다. 게다가 연속된 2 MiB/1 GiB frame을 찾아야 하고, 4 KiB만 필요해도 2 MiB를 차지하는 내부 단편화가 생기며 보호·swap·COW 단위도 거칠어집니다.	큰 페이지는 크기만 줄입니다. 구조를 바꿔야 하고, 그 답이 multi-level table입니다.
8	How a Multi-Level Page Table Works	각 단계는 512개 엔트리를 가진 page 크기 table입니다 (Linux 이름: PGD → PUD → PMD → PTE). 가상 주소 한 개가 단계마다 index를 하나씩 내고, 마지막 PTE가 물리 페이지를 가리키며 offset이 바이트를 고릅니다. 엔트리가 유효하면 아래 단계로 내려가고, 비어 있으면 그 아래 subtree 전체가 존재하지 않습니다.	none 엔트리 하나가 subtree 전체를 대신합니다. 그래서 table은 매핑된 곳에만 생깁니다.
9	x86-64 vs. ARM64 Address Translation	두 아키텍처 모두 48비트 주소, 4 KiB 페이지에서 9+9+9+9+12 비트의 4단계 walk입니다. x86-64는 PML4 → PDPT → PD → PT이고 루트가 CR3, ARM64는 L0 → L3이고 루트가 TTBR0(user)/TTBR1(kernel)입니다. fault 정보는 x86-64가 CR2와 error code, ARM64가 ESR_EL1과 FAR_EL1에 담습니다. 32비트 프로세스는 상위 단계 엔트리 몇 개만 씁니다.	구조는 같고 단계 이름, 루트 레지스터, 커널 매핑 방식, fault 보고 방식이 다릅니다.
10	What is Inside a PTE?	x86-64 PTE는 64비트이며 PFN과 flag 비트로 이루어집니다. P(0)는 존재 여부, RW(1)는 쓰기 허용, U(2)는 user 접근 허용, A(5)는 접근, D(6)는 쓰기, PS(7)는 huge page, G(8)는 global, NX(63)는 실행 금지입니다. ARM64도 개념은 같고 이름만 다릅니다(AP, UXN/PXN, AF, nG).	P = 0이면 매핑이 없다는 뜻이고, 접근 시 page fault가 납니다(demand paging, swap).
11	PTE Flags in Action: Protection and More	MMU는 매 접근마다 flag를 하드웨어로 검사합니다. 쓰기인데 RW = 0, user 모드인데 U = 0, 명령 fetch인데 NX = 1이면 거부되고 #PF가 납니다. P = 0도 #PF이며, 그다음 처리(새 페이지 할당, swap이나 파일에서 읽기, SIGSEGV)는 OS가 정합니다. A는 접근 시, D는 쓰기 시 하드웨어가 켜고, OS는 reclaim과 writeback에 사용합니다.	하드웨어는 막고 알리기만 합니다. 처리는 OS가 합니다.
12	Multi-Level Page Tables: Problems Solved and Trade-offs	8 MiB를 연속으로 매핑하는 예시입니다. 8 MiB ÷ 2 MiB = 4개의 Level-4 table에 Level 1–3 table 각 1개를 더하면 7개 × 4 KiB = 28 KiB이고, flat 512 GiB보다 약 1,900만 배 작습니다. table은 한 페이지라 어디에든 둘 수 있고, 비어 있는 곳에는 table이 없습니다. 대가는 walk가 단계 수만큼 메모리를 읽는다는 점, walker 하드웨어와 TLB가 필요하다는 점, 부분만 쓰는 table 페이지의 낭비입니다.	28 KiB는 8 MiB, 4 KiB 페이지, 48비트 주소 가정에서 나온 예시입니다. 느려지는 walk가 TLB의 이유입니다.
13	What Is a TLB? Hit and Miss	TLB는 MMU 안의 작고 빠른 하드웨어 캐시이며, VPN → PFN과 PTE flag를 저장합니다(PCID/ASID 태그 포함). L1은 작고 병렬로 검색하며 iTLB/dTLB로 나뉘고, L2는 크고 set-associative이며 보통 통합입니다. 검색 순서는 L1 → L2 → page walk입니다. hit이면 PA = PFN + offset으로 끝나고 table을 읽지 않습니다. miss이면 하드웨어 walker가 table을 읽어 채운 뒤 재시도하며, 무효한 엔트리일 때만 page fault입니다.	TLB miss는 page fault가 아닙니다. 반대로 TLB hit이어도 권한 위반이면 fault가 납니다.

쪽	슬라이드	내용 요약	기억할 점
14	TLB Issue: TLB Reach	TLB reach = 엔트리 수 × 페이지 크기로, TLB가 한 번에 매핑할 수 있는 메모리입니다. 예: 엔트리 2,048개 × 4 KiB = 8 MiB. 작업 집합(working set)이 reach보다 작으면 대부분 hit이고, 크면 재사용 전에 쫓겨나 capacity miss가 계속됩니다(1 GiB 무작위 접근의 이상적 hit 비율은 8 MiB / 1 GiB = 0.8%). 해법은 큰 페이지, locality 개선, 하드웨어 보조(L2 TLB, walk cache)입니다.	reach는 KiB~MiB 단위이고 RAM은 GiB 단위입니다. 큰 페이지 하나가 reach를 크게 키웁니다.
15	TLB Issue: TLB Shootdown	TLB는 core마다 있고, 하드웨어는 TLB를 page table이나 다른 core의 TLB와 일치시켜 주지 않습니다. 한 core가 munmap()으로 PTE를 지워 페이지를 해제해도, 다른 core의 TLB에 옛 엔트리가 남아 있으면 이미 다른 용도로 쓰이는 frame에 접근할 수 있습니다. 그래서 PTE 갱신 → 자기 TLB flush → 다른 core에 invalidate 요청(x86은 IPI, Arm64는 broadcast TLBI 명령) → ack 대기 → 페이지 해제 순서로 처리합니다.	PTE를 바꾼 뒤 페이지를 재사용하기 전에 shootdown이 필요합니다. 비용은 core 수에 따라 커지므로 커널이 모아서 처리합니다.
16	TLB Issue: Context Switching	서로 다른 프로세스가 같은 VPN을 쓰므로, 태그가 없으면 context switch마다 TLB를 비워야 합니다. PCID(x86-64)와 ASID(ARM64)를 엔트리에 붙이면 여러 주소 공간의 엔트리가 함께 남아 flush가 줄어듭니다. 커널 매핑은 모든 주소 공간에서 같으므로 global 엔트리(x86 G = 1, Arm64 nG = 0)로 두어 ID 없이 어디서나 쓰이고 flush되지 않습니다.	검색 키는 "ID + VPN"입니다. 사용자 엔트리는 태그가 있고, 커널 엔트리는 global입니다.
17	Kernel and User Mappings: What Is Mapped Where	모든 접근은 가상 주소를 씁니다. 프로세스마다 user 영역 page table을 가지며 frame은 필요할 때 할당됩니다. 커널은 RAM 전체를 "가상 = 물리 + 상수"로 보이게 매핑하고(direct mapping), 커널 이미지는 연속입니다. vmalloc 영역은 가상으로만 연속이고 물리 frame은 흩어져 있을 수 있습니다(vmalloc(), 커널 스택, 모듈).	kmalloc, slab, page allocator는 direct mapping을, vmalloc은 유연한 매핑을 씁니다.
18	Support for Multiple Page Sizes	Linux는 4 KiB, 2 MiB, 1 GiB 페이지를 매핑별로 고릅니다. PUD(1 GiB) 또는 PMD(2 MiB) 엔트리에 PS = 1이면 walk가 거기서 끝나고, 물리 주소 = 물리 base + 페이지 내 offset입니다. 커널은 RAM 매핑에 큰 페이지를, vmalloc에는 4 KiB를 씁니다. 사용자 공간은 기본 4 KiB이고, THP(커널이 가능하면 자동 사용, best-effort, 쪼갠 뒤 swap 가능)나 hugetlb(관리자가 pool을 미리 예약, 보장, swap 안 됨)로 큰 페이지를 얻습니다.	THP는 "되면 쓰는" 방식이고 hugetlb은 "예약해 두는" 방식입니다.
19	Why Multiple Page Sizes? Benefits and Trade-offs	같은 영역을 덮는 데 필요한 번역 수가 다릅니다. 1 GiB는 4 KiB로 262,144개, 2 MiB로 512개, 1 GiB 페이지로 1개입니다. 큰 페이지는 reach가 늘고 TLB miss가 줄며 walk가 짧아지지만, 메모리 낭비, 느린 첫 접근, 높은 할당 비용, 거친 관리 단위가 따릅니다. 작은 페이지는 작거나 희소한 메모리와 많은 프로세스에 맞습니다.	크고 뜨거운 working set(DB buffer, JVM heap, VM guest)에는 큰 페이지가, 그 외에는 4 KiB가 맞습니다.
20	Why the Kernel Runs Fast: The Mapping View	커널이 빠른 것은 매핑 배치 덕분입니다. RAM이 부팅 때 미리 매핑되어 demand fault가 없고, 큰 페이지(2 MiB/1 GiB)로 TLB 엔트리와 walk가 줄며, global 엔트리라 context switch 뒤에도 남고, 모든 프로세스가 같은 커널 매핑을 공유해 walk가 캐시된 PTE에 맞습니다. 하드웨어가 보장하는 것은 아닙니다(MMU는 커널/사용자 주소를 똑같이 다룹니다). vmalloc, 스택, 모듈은 4 KiB로 더 많은 TLB 엔트리를 쓰고, set_memory_ro()는 큰 페이지를 4 KiB로 쪼개며, TLB shootdown IPI 비용도 있습니다.	속도의 출처는 하드웨어가 아니라 커널의 매핑 설계입니다. vmalloc과 쪼개진 매핑은 비용이 더 듭니다.

쪽	슬라이드	내용 요약	기억할 점
21	Observing TLB and Page Walks	perf로 TLB miss와 page walk를 셉니다. 이벤트 이름은 아키텍처에 따라 다릅니다(x86-64: dTLB-load-misses, dtlb_load_misses.walk_completed. ARM64: L1D_TLB_REFILL, L2D_TLB_REFILL, DTLB_WALK, ITLB_WALK). `perf list` 로 확인하고, 많은 VM은 PMU를 노출하지 않습니다. 작업 집합을 키우면 TLB 단계를 넘을 때마다 접근 시간이 계단처럼 오릅니다(Intel Xeon KVM VM에서 얻은 예시이며 CPU마다 다릅니다).	이벤트 이름은 `perf list` 로 확인합니다. 수치는 CPU와 환경에 따라 달라지는 예시입니다.
22	Checkpoint: Virtual to Physical Mapping	1부 정리 문제 6개입니다. multi-level table이 푼 것과 남은 문제, TLB miss와 page fault의 차이, TLB reach, TLB shootdown, context switch와 flush, 여러 페이지 크기를 묻습니다.	먼저 스스로 답해 본 뒤 마지막 장의 Checkpoint 답과 비교합니다.

2부. page fault가 나면 커널은 무엇을 할까요? (24–35쪽)

이 블록의 핵심은 다음과 같습니다. page fault는 오류가 아니라 정상적인 메모리 관리 경로이고, 커널은 VMA를 확인해 페이지를 매핑하거나 할당합니다. 새 물리 frame이 필요한 경우는 일부입니다.

쪽	슬라이드	내용 요약	기억할 점
24	What Causes a Page Fault?	page fault는 두 가지 축으로 나눕니다. PTE의 P 비트로 나누면 not-present(P = 0, 유효한 매핑이 없음)와 permission(P = 1, 매핑은 있으나 접근이 허용되지 않음)입니다. 처리 결과로 나누면 minor(저장장치 I/O 없이 해결: 이미 메모리에 있거나 zero 페이지만 있으면 됨)와 major(저장장치에서 읽어야 하며 보통 수 자릿수 느림)입니다.	"not-present / permission"과 "minor / major"는 서로 다른 기준입니다.
25	Not-present Faults: Anonymous Pages	anonymous 페이지는 파일이 없는 메모리(heap, stack, anonymous mmap)입니다. 첫 접근(minor)에서는 PTE가 없으므로 누락된 page-table 페이지(PUD/PMD/PTE)마다 frame을 하나씩 할당하고, 데이터는 write면 zero로 채운 새 frame, read면 공유 zero page를 줍니다. 마지막으로 PTE를 설정하고 재시도합니다. swap-out된 페이지(major)는 PTE에 swap entry가 있어 swap 장치에서 읽어 와야 하며, swap cache에 있으면 I/O 없이 minor로 끝납니다.	anonymous 페이지는 첫 접근에서 만들어지거나(minor) swap에서 복원됩니다(major).
26	Not-present Faults: File-backed Pages	file-backed 페이지는 실행 파일, 공유 라이브러리, mmap된 파일의 데이터이며 page cache에 있습니다. page cache에 있으면(hit) 새 데이터 frame 없이 cached frame만 매핑하고 I/O도 없습니다(minor). fault-around는 근처 cached 페이지도 함께 매핑합니다. 없으면(miss) 저장장치에서 새 page-cache frame으로 읽어 온 뒤 매핑합니다(major, read-ahead 가능). MAP_PRIVATE 매핑의 첫 접근이 write이면 fault 한 번에 파일 페이지를 복사합니다(COW).	cache hit이면 minor, miss이면 major입니다.
27	Not-present Faults: Other Cases	모든 not-present fault가 페이지를 매핑하는 것은 아닙니다. 스택 VMA 바로 아래 주소는 한도 안에서 VMA를 늘린 뒤 anonymous 첫 접근처럼 처리합니다. tmpfs, memfd, System V shared memory는 RAM 기반이라 첫 접근이 minor이고 swap-out 뒤에는 major입니다. 어떤 VMA에도 속하지 않는 주소(NULL, 해제된 메모리)는 아무것도 할당하지 않고 SIGSEGV입니다. system call이 user 버퍼를 읽거나 쓰다가(copy_to_user 등) 나는 kernel-mode fault는 user fault처럼 처리하고, 잘못된 주소이면 -EFAULT를 돌려줍니다.	not-present fault는 매핑, 스택 확장, shmem 처리, SIGSEGV 중 하나로 끝납니다.

쪽	슬라이드	내용 요약	기억할 점
28	Permission Faults: Permission Bits	P = 1이지만 PTE 권한이 접근을 막은 경우입니다(RW = 0인데 write, U = 0인데 user 접근, NX = 1인데 명령 fetch). MMU는 fault만 보고하고 VMA 정책은 모릅니다. 커널이 VMA를 확인해서, VMA가 거부하면 SIGSEGV를 보냅니다. VMA가 허용하면 정당한 fault이며 커널이 PTE를 고치고 재시도합니다. 예: fork 뒤 COW(부모와 자식이 read-only로 공유하다가 write에서 복사), read 뒤 첫 write(읽기가 공유 zero page를 read-only로 매핑했다가, write에서 새 frame 할당).	permission fault는 대개 SIGSEGV이지만, VMA가 허용하면 COW처럼 정상 동작입니다.
29	Summary of Page Fault Handling Flow	처리 흐름은 네 단계입니다. 1) Exception: #PF로 커널에 진입하며 CR2에 fault 주소, error code가 저장됩니다. 2) Validate: 주소가 속한 VMA를 찾고 접근이 허용되는지 확인하며, 아니면 SIGSEGV입니다. 3) Resolve: 페이지를 매핑, 할당, 읽기 또는 PTE 수정으로 사용 가능하게 만듭니다. 4) Resume: PTE를 갱신하고 명령을 다시 실행합니다.	대부분의 fault는 오류가 아닌 정상 동작입니다.
30	Physical Memory: Page Frames and PFN	물리 메모리는 고정 크기 frame(PAGE_SIZE, 보통 4 KiB)으로 나뉘고 frame마다 고유한 PFN이 있습니다. PFN = PA >> 12, PA = (PFN << 12) | offset입니다. 예: PA 0x0010_1ABC는 PFN 0x101, offset 0xABC입니다. 주소 공간에는 메모리가 없는 hole도 있습니다. frame의 상태는 struct page에 저장합니다.	PFN은 물리 주소에서 하위 12비트(offset)를 뺀 번호입니다.
31	Struct Page: Physical Page Metadata	커널은 물리 page frame마다 struct page를 하나 두고 mem_map[] 배열에 PFN 순서로 보관합니다(pfn_to_page(pfn) = &mem_map[pfn]). 상태/flag, 참조 횟수(page table, page cache, slab 등), mapping 정보(anonymous, page cache, 파일)를 담습니다. buddy allocator, page cache, SLUB, reclaim/LRU가 사용합니다. 정확한 구조는 커널 버전마다 다르지만 역할은 같습니다.	struct page는 물리 페이지 하나당 메타데이터 하나입니다. 필드 이름은 몰라도 역할만 알면 됩니다.
32	Physical Memory Zones	Linux는 물리 메모리를 주소 범위와 제약에 따라 zone으로 나눕니다. x86-64의 전형적인 구성은 DMA(0–16 MB, 오래된 DMA 장치), DMA32(16 MB–4 GB, 32비트 DMA 장치), Normal(4 GB 초과, 일반 용도)입니다. 장치가 낮은 물리 주소만 접근할 수 있고, 제약이 다른 페이지를 분리해 두려는 정책 때문입니다. zone은 allocation flag(예: GFP_DMA)로 고르며, 구성은 아키텍처와 커널 버전에 따라 다릅니다.	zone은 "이 장치/용도가 쓸 수 있는 주소 범위"로 나눈 구역입니다.
33	Page Frame Allocation after a Page Fault	새 frame이 필요한 경우는 anonymous 첫 접근(stack, shmem 포함), COW 복사, page cache miss, swap-in, page-table 페이지입니다. 필요 없는 경우는 page cache hit, swap cache hit, anonymous read 첫 접근(공유 zero page 매핑, 이후 write에서 할당)입니다. frame은 allocator가 2^order 블록으로 주기도 하고, read-ahead나 fault-around, THP(512 frame)처럼 한 번에 여럿이 필요한 경우도 있습니다. free frame이 부족하면 watermark 아래에서는 reclaim(Week 9), cgroup 한도에서는 cgroup 안에서 reclaim합니다.	fault가 곧 할당은 아닙니다. 페이지를 새로 만들거나 복사할 때만 frame이 필요합니다.

쪽	슬라이드	내용 요약	기억할 점
34	Observing Minor / Major Page Faults	카운터는 page-faults = minor + major입니다. 예: malloc(100 MB)만 하면 minor 57회, 4 KiB마다 1바이트씩 쓰면 25,656회로, 차이 25,599는 100 MB ÷ 4 KiB = 25,600 페이지에 해당합니다(57회는 시작 시 프로그램과 라이브러리 fault). 64 MB 파일(16,384 페이지)을 모두 접근하면 cold + read-ahead off는 major 16,384회(1.16초), warm은 0회(0.010초), cold + read-ahead on은 1회(약 0.1초)였습니다. 도구는 `perf stat -e page-faults`, `/proc/<pid>/stat`, `/usr/bin/time -v` 입니다.	수치는 Xeon 2.8 GHz 클라우드 VM, Linux 6.x에서 얻은 예시이며 버전과 환경에 따라 다릅니다.
35	Checkpoint: Page Fault vs Physical Page Allocation	2부 정리 문제입니다. 시나리오 8개에 대해 P 비트, minor/major, 새 frame 할당 여부를 판단합니다.	먼저 스스로 답해 본 뒤 마지막 장의 Checkpoint 답과 비교합니다.

3부. 커널은 물리 메모리를 어떻게 나눠 줄까요? (37–54쪽)

이 블록의 핵심은 다음과 같습니다. buddy allocator가 zone, order, migratetype으로 물리 페이지를 관리하고, SLUB가 그 페이지를 작은 객체로 나눕니다. 어느 API를 쓸지는 크기, 연속성, 문맥으로 정합니다.

쪽	슬라이드	내용 요약	기억할 점
37	Linux Kernel Memory Allocators: Landscape	사용자 공간에서는 malloc()이 brk()/mmap()으로 가상 메모리(VMA)를 얻고, 첫 접근에서 page fault가 나면 물리 페이지가 할당됩니다. 커널에서는 kmalloc()이 SLUB를 쓰고, vmalloc()은 흩어진 페이지를 가상으로 이어 붙이며 SLUB를 거치지 않습니다. page fault, 새 slab, vmalloc 매핑은 모두 buddy allocator에서 끝납니다.	서로 다른 allocator가 있지만 물리 페이지는 결국 buddy allocator가 줍니다.
38	Buddy Allocator Overview	buddy allocator는 zone마다 2^n개의 연속 페이지 블록으로 물리 메모리를 관리합니다. 할당할 때는 큰 블록을 쪼개 필요한 크기를 만들고(split), 해제할 때는 free buddy와 합칩니다(merge). zone은 block 크기(order)마다 free list를 하나씩 두고, 이는 나중에 migratetype별로 다시 나뉩니다 (`free_area[order].free_list[type]`). order로 리스트를 찾고 buddy는 주소 연산으로 찾으므로 빠릅니다.	할당은 쪼개고, 해제는 합칩니다. 합쳐야 큰 블록이 다시 생깁니다.
39	What Is an Order?	order n은 2^n개의 연속 페이지 블록입니다. order 0 = 1페이지(4 KiB), order 3 = 8페이지(32 KiB), order 10 = 1,024페이지(4 MiB, 전형적인 최대). 호출자가 바이트 수가 아니라 order를 넘기며, N페이지가 필요하면 충분히 큰 가장 작은 order(order = ceil(log2 N), `get_order(N * PAGE_SIZE)`)를 고릅니다. 예: 5페이지 → order 3(8페이지, 3페이지 낭비).	요청한 order가 검색할 free list를 정합니다. order가 크면 남는 페이지가 생깁니다.
40	Buddy Allocation API	핵심 호출은 `alloc_pages(gfp_mask, order)` 로, 2^order 연속 페이지 블록의 첫 페이지를 돌려주고 실패하면 NULL입니다. `alloc_page()` , `__get_free_pages()` , `get_zeroed_page()` 는 이를 바탕으로 한 도우미이고, 해제는 같은 order로 `__free_pages()` 를 씁니다. gfp_mask는 zone과 sleep 가능 여부를 정합니다. GFP_KERNEL은 sleep할 수 있고(process context), GFP_ATOMIC은 sleep할 수 없어 실패할 수 있습니다.	atomic 문맥에서 sleep하는 flag를 쓰면 커널이 문제를 일으킬 수 있습니다.

쪽	슬라이드	내용 요약	기억할 점
41	Buddy Allocator: Splitting and Coalescing	정확한 크기의 블록이 없으면 더 큰 블록을 두 buddy로 쪼개고, 남은 buddy는 한 단계 낮은 order의 free list에 넣습니다. 해제할 때는 free buddy와 합쳐 order를 한 단계 올립니다. buddy 주소는 한 비트를 뒤집어 찾습니다(`buddy = P XOR (1 << (N + PAGE_SHIFT))`). 인접하고 free이며 같은 zone에 있을 때만 합쳐집니다.	buddy를 찾는 것은 주소 연산 한 번이라 빠릅니다.
42	Problem of Buddy Allocator: External Fragmentation	free 페이지는 충분한데 연속된 high-order 블록이 없어 요청이 실패하는 것이 외부 단편화입니다. 서로 인접한 free 블록이라도 buddy 관계가 아니면 합쳐지지 않습니다. huge page, 큰 DMA 버퍼처럼 큰 연속 메모리가 필요한 작업이 영향을 받습니다. `/proc/buddyinfo` 에서 order별 free 블록 수로 볼 수 있으며, 총 free 페이지 수가 같아도(예: 526개) 분포가 다르면 order 3 이상 블록이 하나도 없을 수 있습니다.	개수가 아니라 order별 분포를 비교합니다.
43	Solutions: Migratetypes	메모리를 pageblock으로 나누고 pageblock마다 migratetype 태그(Unmovable, Movable, Reclaimable)를 하나 붙입니다. 부팅 때는 모두 movable입니다. 예: Unmovable은 slab과 page table, Movable은 사용자 페이지와 page cache, Reclaimable은 dentry와 inode입니다. order마다 type별 free list가 있고, gfp_mask(예: __GFP_MOVABLE, __GFP_RECLAIMABLE)가 type을 정합니다. 해당 type에 블록이 없으면 다른 type의 가장 큰 블록을 가져오며 pageblock 태그가 바뀔 수 있습니다. 규칙은 커널 버전마다 다릅니다.	이동할 수 없는 페이지를 한곳에 모아두면, 나머지 pageblock을 비워 큰 블록을 만들 수 있습니다.
44	Solutions: Compaction	compaction은 부분만 쓰이는 pageblock에 흩어진 free 페이지를 모으는 작업입니다. movable 페이지를 한쪽 끝으로 migrate하면 movable만 있던 pageblock이 통째로 free 블록이 됩니다. unmovable 페이지가 하나라도 있는 pageblock은 비울 수 없습니다. migratetype이 unmovable을 적은 수의 pageblock에 모아 두므로 더 많은 pageblock을 비울 수 있습니다. 대가는 page migration이 CPU 시간과 메모리 대역폭을 쓴다는 점입니다.	compaction이 언제 시작되는지는 Week 9에서 다룹니다.
45	Contiguous Memory for Devices: CMA	카메라, 코덱, 디스플레이 같은 장치는 수 MiB의 물리 연속 버퍼가 필요합니다. buddy에는 커널 설정에 따른 최대 order가 있고, 단편화로 큰 블록이 없을 수도 있으며, 영구 예약은 idle일 때 낭비입니다. CMA는 부팅 때 영역을 예약해 두고, idle 동안에는 movable 페이지만 쓰게 합니다. `cma_alloc()` 이 그 페이지들을 옮겨 내고 연속 버퍼를 만들며, 드라이버가 `dma_alloc_coherent()` 로 DMA 버퍼를 요청할 때 호출되기도 합니다. pinned 페이지는 옮길 수 없어 다른 범위를 시도하다 NULL을 돌려줄 수 있고, migration은 복사라 지연이 생깁니다.	크기는 설정값입니다(커널 기본 약 16 MiB). `/proc/meminfo` 의 CmaTotal로 확인합니다.
46	Buddy Allocator: Allocation Path (Simplified)	할당 경로는 fast path(low watermark 위), slow path(reclaim, compaction), failure(OOM killer, low-memory killer)의 세 단계입니다. fast path는 gfp_mask로 zone을 고르고, free 페이지가 low watermark 위인지 확인하고(아니면 다음 zone), 요청 order의 `free_area[order]` 에서 블록을 꺼내며 필요하면 더 큰 블록을 쪼개 돌려줍니다. watermark 아래이거나 요청 order의 블록이 없으면(외부 단편화) slow path로 갑니다. 단순화한 경로이며 Linux는 작은 페이지 할당을 빠르게 하려고 per-CPU page cache도 씁니다.	slow path는 Week 9, OOM과 복구는 Week 10에서 다룹니다.

쪽	슬라이드	내용 요약	기억할 점
47	Observing the Buddy Allocator	`/proc/buddyinfo` 는 zone과 order별 free 블록 수(열 n = order n, order 10 = 4 MiB), `/proc/zoneinfo` 는 zone별 min/low/high watermark(페이지 단위), `/proc/pagetypeinfo` 는 migratetype별 free 블록과 pageblock 수를 보여 줍니다. `/proc/meminfo` 에는 CMA가 켜져 있으면 CmaTotal과 CmaFree가 있습니다. 단편화를 보려면 기준선이 필요하며, 고차 블록이 차지하는 free 페이지 비율을 시간에 따라 비교합니다.	예시 출력은 Linux 6.18, x86-64 VM(CMA 꺼짐, pageblock order 9)입니다. 값은 환경마다 다릅니다.
48	Why is Buddy Allocator Not Enough?	buddy는 페이지 단위로 줍니다. 128 B 객체 하나에 4 KiB 페이지를 주면 96% 이상이 낭비됩니다. 커널 객체는 task_struct 약 2–6 KiB, inode 약 600 B, dentry 약 192 B, file 약 256 B, sk_buff 약 256 B, timer_list 약 64 B처럼 수십~수천 바이트이고(크기는 근사치) 매우 자주 할당·해제됩니다. 메모리 낭비, 빈번한 페이지 할당의 오버헤드와 락, 수백만 객체의 관리라는 세 문제가 있어 전용 allocator가 필요합니다.	작은 객체에는 SLUB를 씁니다.
49	SLUB Allocator Overview	slab은 연속된 페이지를 같은 크기의 객체로 나눈 것이고, cache는 한 객체 크기나 타입에 대한 slab들의 집합입니다. 각 slab은 buddy allocator에서 연속 페이지를 받으며, 서로 다른 slab의 페이지가 인접할 필요는 없습니다. 해제된 객체는 slab에 남아 다음 할당에 재사용됩니다. 예: kmalloc-64는 1페이지에 64개, kmalloc-256은 1페이지에 16개, task_struct는 8페이지에 5개입니다. 예전에는 SLAB과 SLOB도 있었고, 현재 Linux는 단순한 메타데이터와 다중 코어 확장성이 좋은 SLUB를 씁니다.	cache → slab → object 순서로 기억합니다.
50	kmalloc() and kfree(): Generic Caches	`kmalloc(size, gfp)` 는 kmalloc-64, kmalloc-256 같은 일반 size class cache에서 객체를 주며, 요청 크기를 다음 cache 크기로 올립니다. 가장 큰 cache(전형적으로 8 KiB)를 넘는 요청은 buddy allocator가 처리합니다. 최대 크기는 커널 설정과 아키텍처에 따라 다르고, 메모리는 물리적으로 연속입니다. 더 크거나 물리 연속이 필요 없으면 `vmalloc()` 을 씁니다. 해제는 `kfree()` 입니다. task_struct, inode, dentry는 일반 cache가 아니라 전용 cache(kmem_cache)를 씁니다.	예시: kmalloc-64는 객체 64 B, slab당 64개, 1페이지입니다. 값은 커널 버전마다 다릅니다.
51	kmem_cache: Dedicated Caches	`kmem_cache_create(name, size, align, flags, ctor)` 로 cache를 만들고, `kmem_cache_alloc(cache, GFP_KERNEL)` 로 할당하고 `kmem_cache_free()` 로 해제하며 `kmem_cache_destroy()` 로 없앱니다. 전용 cache는 정확한 크기(반올림 낭비 없음), constructor(slab을 만들 때 객체마다 한 번 초기화), `/proc/slabinfo` 통계를 제공합니다. 예: task_struct가 5,888 B일 때 `kmalloc(5888)` 은 kmalloc-8k의 8,192 B 객체를 써서 2,304 B가 낭비됩니다.	예시(Linux 6.18, x86-64): task_struct 5,888 B/slab당 5개/8페이지, dentry 192 B/21개/1페이지. 값은 환경마다 다릅니다.
52	Choosing a Kernel Allocation API	`kmalloc()` / `kzalloc()` 은 bytes~KiB의 작은 객체를 물리 연속으로 주며 SLUB를 쓰고(큰 요청은 buddy), kzalloc은 0으로 채웁니다. `alloc_pages()` 는 페이지 배수의 물리 연속 블록(최대 order까지)을 buddy에서 받아 `struct page *` 를 돌려주고, 0 초기화는 `__GFP_ZERO` 로 합니다. `vmalloc()` 은 큰 크기의 가상 연속 메모리(물리는 흩어질 수 있음)이며 `vzalloc()` 으로 0 초기화합니다. 크기, 연속성, 문맥으로 고릅니다.	작은 객체는 kmalloc, 연속 페이지는 alloc_pages, 큰 가상 연속 영역은 vmalloc입니다.

쪽	슬라이드	내용 요약	기억할 점
53	Observing Slab Memory: slabinfo and meminfo	`/proc/slabinfo` (root 필요)는 cache별로 active(사용 중)와 num(할당된) 객체 수, 객체 크기, slab당 객체 수와 페이지 수, slab 수를 보여 줍니다. cache가 쓰는 메모리 = slab 수 × slab당 페이지 수 × 페이지 크기이며, 예를 들어 task_struct는 28 × 8 × 4 KiB = 896 KiB입니다. `slabtop -o -s c` 는 cache 크기순으로 정렬합니다. `/proc/meminfo` 의 Slab은 SReclaimable(dentry, inode cache 등, 메모리 압박 때 해제 가능: Week 9)과 SUnreclaim의 합입니다. 서로 다른 카운터라 slabinfo와 값이 정확히 같을 필요는 없습니다.	예시 환경은 Linux 6.18, x86-64 VM입니다. 값은 커널 버전, 설정, 워크로드에 따라 다릅니다.
54	Checkpoint: Which Allocator?	3부 정리 문제입니다. 시나리오 6개에 대해 크기, 연속성, 문맥을 보고 알맞은 할당 API와 GFP flag를 고릅니다.	먼저 스스로 답해 본 뒤 마지막 장의 Checkpoint 답과 비교합니다. 정답이 하나만 있는 문제는 아닙니다.

메모리를 줄일 때 (55쪽)

이 블록의 핵심은 다음과 같습니다. 이번 주에 배운 구조(프로세스 메모리, page cache, 커널 메모리, CMA)에서 줄일 수 있는 곳과 그 대가를 정리합니다.

쪽	슬라이드	내용 요약	기억할 점
55	Reducing Memory Use: Targets and Trade-offs	메모리를 줄일 대상을 프로세스 메모리(heap, stack), 파일 캐시(page cache), 커널 메모리, CMA·예약 메모리, 여유 메모리 headroom(`vm.min_free_kbytes`)으로 나누고, 방법과 대가(trade-off), 주의할 점을 표로 정리합니다. 예: 프로세스 메모리는 버퍼를 줄이거나 필요할 때 읽어 오고 라이브러리를 공유해서 줄이며, 대가는 나중의 CPU나 I/O 작업입니다. swap이 없으면 모든 서비스의 최대 사용량이 RAM에 들어가야 하고 PSS(`/proc/<pid>/smaps`)로 확인합니다. 파일 캐시는 clean 상태이면 swap 없이도 해제할 수 있지만, 너무 줄이면 같은 데이터를 반복해서 읽습니다. 커널 메모리는 `Slab` , `PageTables` , `KernelStack` , CMA는 `CmaTotal` 을 `/proc/meminfo` 에서 확인합니다.	줄이는 모든 방법에는 대가가 있습니다. 타깃 기기에서 전후를 측정한 뒤에만 바꿉니다.

Week 8 한 장 요약과 핵심 용어 (56쪽)

한 줄 요약. 가상 주소는 page table walk와 TLB를 거쳐 물리 주소가 되고, 매핑이 없거나 권한이 없으면 page fault로 커널이 처리합니다. 커널은 buddy allocator로 물리 페이지를, SLUB로 작은 객체를 할당합니다. 메모리가 부족해질 때의 일(reclaim, swap, OOM)은 다음 주부터입니다.

쪽	슬라이드	내용 요약	기억할 점
56	Week 8 Summary	세 칸으로 정리합니다. 1) 주소 변환: multi-level page table과 TLB, 페이지 크기는 TLB reach와 관리 단위의 균형입니다. 2) 물리 페이지 할당: page fault에서 커널이 페이지를 할당하거나 매핑하고 PTE를 갱신해 재시도하며, buddy allocator가 zone, order, migratetype, watermark로 관리합니다. 3) 커널 객체 할당: SLUB의 kmalloc과 kmem_cache, slab, freelist입니다. 메모리 예산은 trade-off가 감수할 만한 곳에서만 줄이고 타깃에서 측정합니다. 다음 시간(Week 9)은 memory pressure와 page reclaim입니다.	주소 변환 → fault → 물리 페이지 → 객체 할당의 흐름으로 기억합니다.

용어	의미
page, frame	가상 공간의 고정 크기 단위(page)와 물리 메모리의 같은 크기 단위(frame). 보통 4 KiB
VPN, PFN	가상 페이지 번호, 물리 페이지 프레임 번호(PFN = PA >> 12)
page table, PTE	VPN을 PFN에 대응시키는 프로세스별 table과 그 엔트리(PFN + flag)
P, RW, U, NX, A, D	PTE flag: 존재, 쓰기 허용, user 접근, 실행 금지, 접근됨, 수정됨
PGD, PUD, PMD, PTE	Linux의 4단계 page table 이름(x86-64: PML4, PDPT, PD, PT)
CR3, TTBR	현재 주소 공간의 page table 루트를 가리키는 레지스터(x86-64, ARM64)
page walk	TLB miss 때 하드웨어가 page table을 단계별로 읽는 과정
TLB	주소 번역 결과를 캐시하는 MMU 안의 하드웨어 캐시
TLB reach	엔트리 수 × 페이지 크기. TLB가 한 번에 매핑하는 메모리
TLB shootdown	PTE를 바꾼 뒤 다른 core의 TLB 엔트리를 무효화하는 과정
PCID, ASID	TLB 엔트리에 붙여 주소 공간을 구분하는 태그(x86-64, ARM64)
THP, hugetlb	큰 페이지를 얻는 두 방법. THP는 best-effort이고 hugetlb은 미리 예약하며 swap되지 않음
page fault	매핑이 없거나 권한이 없는 접근에서 발생하는 예외. 커널이 VMA를 보고 처리
not-present, permission	P = 0인 fault와 P = 1이지만 권한이 거부된 fault
minor, major fault	저장장치 I/O 없이 해결되는 fault와 I/O가 필요한 fault
COW	copy-on-write. 공유 페이지에 쓸 때 복사본을 만드는 방식
zero page	한 번도 쓰지 않은 anonymous 페이지를 읽을 때 매핑하는 공유 read-only 페이지
page cache	파일 내용을 담는 메모리 캐시
struct page	물리 page frame마다 하나씩 있는 커널 메타데이터
zone	주소 범위와 제약으로 나눈 물리 메모리 구역(DMA, DMA32, Normal)
buddy allocator	2^order 연속 페이지 블록을 쪼개고 합치며 관리하는 물리 페이지 allocator
order	블록 크기 2^order 페이지를 나타내는 번호
migratetype, pageblock	pageblock에 붙는 이동 가능성 태그(Unmovable, Movable, Reclaimable)와 그 단위
compaction	movable 페이지를 옮겨 연속 free 블록을 만드는 작업
CMA	장치용 연속 버퍼를 위해 예약해 두고 idle 때 movable 페이지가 쓰는 영역
watermark	zone의 min/low/high free 페이지 기준. fast path는 low 위일 때 성공
gfp_mask	할당 flag. zone과 sleep 가능 여부 등을 정함(GFP_KERNEL, GFP_ATOMIC)
slab, cache, SLUB	같은 크기 객체 묶음, 그 집합, 현재 Linux의 slab 구현
kmalloc, kmem_cache	일반 size class로 할당하는 API와 객체 타입 전용 cache API
vmalloc	가상으로만 연속인 큰 영역을 할당하는 API

용어	의미
SReclaimable, SUnreclaim	Slab 중 메모리 압박 때 해제할 수 있는 부분과 없는 부분

Checkpoint 문제와 답 (22, 35, 54쪽)

먼저 슬라이드의 질문을 보고 스스로 답해 본 뒤, 아래 답과 비교해 보시기 바랍니다.

22쪽. Virtual to Physical Mapping

질문	답
multi-level page table이 푼 문제와 남은 문제는 무엇일까요?	푼 것은 크기입니다(flat 512 GiB vs. 매핑된 페이지 하나에 16 KiB). 남은 문제는 속도입니다. 한 번 miss할 때마다 page walk로 메모리를 4번 읽습니다.
TLB miss는 모두 page fault일까요?	아닙니다. miss이고 PTE가 유효하면 하드웨어가 page walk로 채웁니다. P = 0이거나 권한 위반일 때만 page fault이며, TLB hit이어도 권한 위반이면 fault가 날 수 있습니다.
TLB reach란 무엇이고 왜 중요할까요?	reach = 엔트리 수 × 페이지 크기입니다. working set이 reach보다 크면 miss와 walk가 늘어납니다. 2 MiB 페이지는 reach를 512배로 만듭니다.
TLB shootdown은 왜 필요할까요?	TLB는 core마다 있고 서로 일치시켜 주지 않아서 다른 core가 옛 엔트리를 가질 수 있습니다. PTE 갱신 → 자기 TLB flush → IPI → ack → 페이지 해제 순서로 처리합니다.
context switch에서 TLB flush를 피하려면 어떻게 할까요?	엔트리에 PCID/ASID를 붙여 프로세스마다 자기 엔트리에만 맞게 하고, 커널 엔트리는 global로 두어 switch 뒤에도 남깁니다.
여러 페이지 크기는 어떻게 지원할까요?	walk를 일찍 끝냅니다(PMD에서 2 MiB, PUD에서 1 GiB, PS = 1). THP는 best-effort이고 swap할 수 있으며, hugetlb은 예약이고 swap되지 않습니다.

마무리 질문: 가상 주소가 물리 주소가 되는 과정과, TLB miss와 page fault의 차이를 설명할 수 있나요?

35쪽. Page Fault vs Physical Page Allocation

각 시나리오에서 P 비트, minor/major, 새 frame 할당 여부를 답합니다.

시나리오	P 비트 / 유형 / 새 frame
anonymous 페이지를 처음 write	P = 0 / minor / 새 frame 필요 (zero로 채운 frame)
anonymous 페이지를 처음 read	P = 0 / minor / 새 frame 없음 (공유 zero page 매핑)
page cache에 있지만 아직 매핑되지 않은 파일 페이지	P = 0 / minor / 새 data frame 없음 (cached frame 매핑)
page cache에 없는 파일 페이지	P = 0 / major / 새 frame 필요 (저장장치에서 새 page-cache frame으로 읽음)
swap-out된 anonymous 페이지	P = 0 / major / 새 frame 필요 (swap에서 읽음)
exec() 뒤 private 파일 매핑(예: .data)에 처음 write	P = 0 / minor 또는 major / 새 frame 필요 (파일 페이지를 얻은 뒤 복사: COW). page cache에 있으면 minor, 없으면 major
먼저 read한 페이지(zero page 등)에 write	P = 1(permission fault) / minor / 새 frame 필요 (VMA가 허용하므로 복사: COW)
read-only 영역(예: 코드)에 write	P = 1 / SIGSEGV / VMA가 거부하므로 프로세스가 SIGSEGV를 받음

마무리 질문: page fault가 항상 새 물리 페이지 할당을 뜻하지는 않는 이유를 설명할 수 있나요?

54쪽. Which Allocator?

시나리오	권장 API와 이유
네트워크 패킷 데이터용 작은 버퍼 (약 1 KB, process context)	`kmalloc(size, GFP_KERNEL)`. 작고 물리 연속이며 일반 문맥입니다.
작은 구조체를 할당하고 0으로 초기화 (128 B, process context)	`kzalloc(size, GFP_KERNEL)`. 0으로 채워야 하는 작은 객체입니다.
legacy DMA용 8개 연속 페이지 (32 KB, DMA zone, 16 MB 미만)	`alloc_pages(GFP_DMA, 3)`. 8페이지 = order 3이고 DMA zone을 지정합니다.
임시 커널 데이터용 큰 버퍼 (10 MB, 가상 연속이면 충분)	`vmalloc(size)`. 크고 물리 연속이 필요 없습니다.
interrupt handler 안의 메모리 (256 B, sleep 불가)	`kmalloc(size, GFP_ATOMIC)`. atomic 문맥이라 sleep하면 안 됩니다.
0으로 초기화된 페이지 1개 (4 KB, process context)	`get_zeroed_page(GFP_KERNEL)`. 페이지 하나를 0으로 채워 줍니다.

마무리 질문: 크기, 연속성, 문맥에 따라 커널 할당 API를 고르는 기준을 설명할 수 있나요?

정답이 하나만 있는 문제는 아닙니다. 선택의 이유(크기, 연속성, 문맥)를 설명할 수 있으면 됩니다.